

Overall Project Structure

The ZIP contains **1 main folder with 7 files**:

Using-Machine-Learning-to-Predict-Alzheimer-s-Disease-main/

|

├— README.md

|

├— Convolutional Neural Network.ipynb

├— Ensemble Methods.ipynb

├— Grad CAM.ipynb

├— KNN Classifier.ipynb

├— Multilayer Perceptron.ipynb

└— SVM_Alzheimer.ipynb

There are **no CSV files, Python .py files, saved datasets, or model files included in the ZIP**.

1. README.md

This is the project documentation.

It explains that the project uses the **OASIS brain MRI dataset** to classify Alzheimer's/dementia stages.

The four classes used are:

MildDemented

ModerateDemented

NonDemented

VeryMildDemented

The README says the project explores:

- K-Nearest Neighbors (KNN)
- Ensemble Methods
- Multilayer Perceptron (MLP)

- Convolutional Neural Network (CNN)
- Support Vector Machine (SVM)
- Class Activation Mapping / Grad-CAM

It reports a **best accuracy of 98.07%**.

2. 🧠 Convolutional Neural Network.ipynb

This notebook implements the main **CNN image-classification approach**.

Basic flow

MRI Images

↓

Load Images using OpenCV

↓

Convert to Grayscale

↓

Resize to 128×128

↓

Assign 4 Dementia Classes

↓

5-Fold Stratified Cross Validation

↓

CNN

↓

Training

↓

Validation

↓

Accuracy / Loss

↓

Confusion Matrix

The notebook uses libraries such as:

numpy

opencv

tensorflow / keras

sklearn

matplotlib

The CNN uses layers such as:

- Conv2D
- MaxPooling2D
- Flatten
- Dense
- Dropout

It also performs **5-fold cross-validation**.

So this is essentially the project's core deep-learning model.

3. 🌳 Ensemble Methods.ipynb

This notebook experiments with **traditional machine-learning ensemble methods**.

The images are first loaded and converted into numerical arrays.

The structure is roughly:

MRI Images

↓

Preprocessing

↓

128 × 128 Images

↓

Flatten Images

↓

Traditional ML Classifiers

↓

Prediction

↓

Accuracy / Confusion Matrix

The notebook includes **Decision Tree-based classification** and additional classifier experimentation.

It also uses **5-fold cross-validation**.

4. 🔥 Grad CAM.ipynb

This is an **explainability notebook**.

Grad-CAM stands for **Gradient-weighted Class Activation Mapping**.

Its purpose is different from simply training a model.

Instead, it tries to answer:

"Which areas of the MRI image influenced the CNN's prediction?"

The flow is approximately:

MRI Image

↓

Trained CNN

↓

Prediction

↓

Gradient Calculation

↓

Activation Map

↓

Heatmap

↓

Overlay on MRI

Importantly, the notebook says it does **not necessarily retrain the model from scratch** for the Grad-CAM portion. It loads a previously trained model:

alzheimers_CNN_aug_model.h5

However, that .h5 model file is **not included in this ZIP**.

So the Grad-CAM notebook may not run successfully as-is unless that model file is provided.

5. 🧠 KNN Classifier.ipynb

This notebook implements **K-Nearest Neighbors**.

One interesting thing here is that it creates a combined dataset structure.

It references:

main_data/

|

├─ train/

├─ test/

└─ combined_dataset/

And inside the combined dataset:

combined_dataset/

|

├─ MildDemented/

├─ VeryMildDemented/

├─ ModerateDemented/

└─ NonDemented/

The notebook also performs:

- Image loading
- Image resizing
- Data inspection

- Image counting
- Data augmentation
- Resampling
- KNN classification

Because KNN operates on numerical features, the image data is transformed into numerical arrays.

6. Multilayer Perceptron.ipynb

This notebook implements an **MLP (Multilayer Perceptron)**.

The interesting part is that it uses **PyTorch** for the MLP section.

The flow is:

MRI Images

↓

Image preprocessing

↓

Train / Validation split

↓

Convert NumPy → PyTorch Tensor

↓

TensorDataset

↓

DataLoader

↓

MLP

↓

Classification

It uses:

torch

torch.nn

torch.utils.data

tqdm

So this notebook mixes the earlier TensorFlow/OpenCV preprocessing with a PyTorch-based neural network.

7. SVM_Alzheimer.ipynb

This is another major machine-learning approach.

It contains several experiments.

Experiment 1 — VGG16 + SVM

The notebook uses:

MRI images

↓

VGG16

↓

Feature extraction

↓

SVM

↓

Classification

It experiments with SVM kernels including:

RBF

Polynomial

Experiment 2 — Custom CNN + SVM

It also explores:

MRI

↓

Custom CNN

↓

Feature extraction

↓

SVM

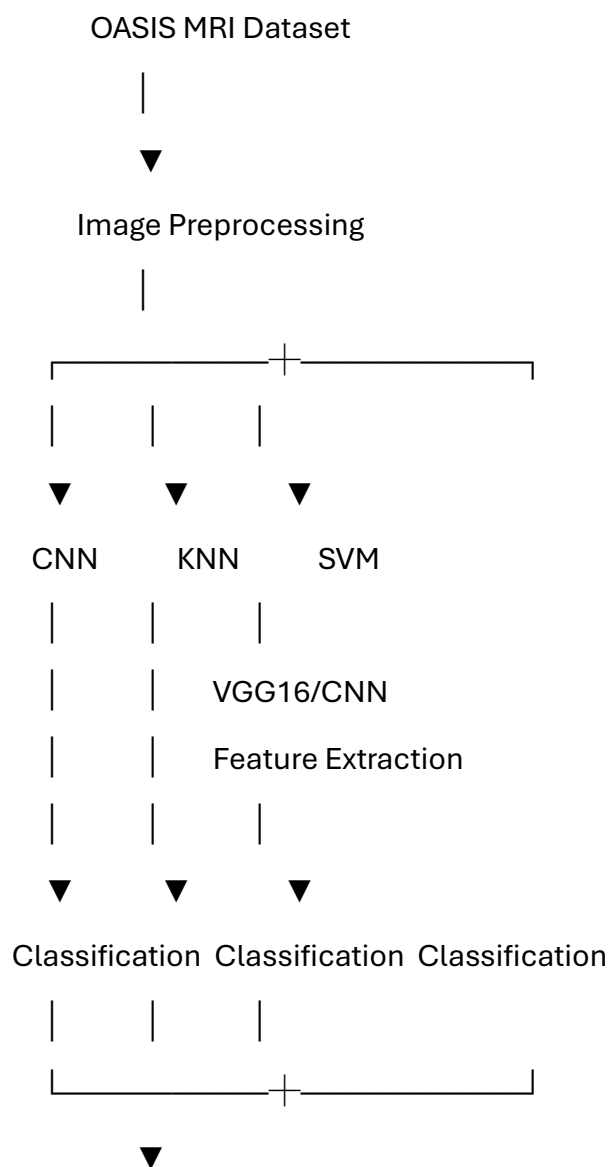
↓

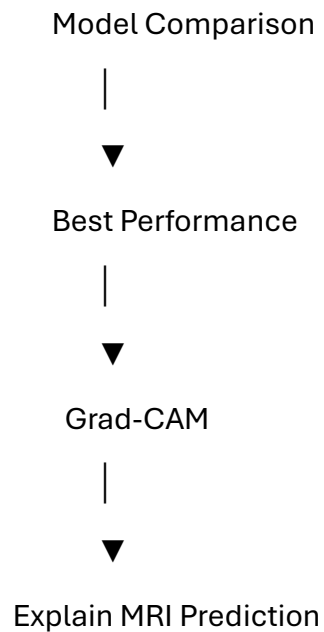
Prediction

The notebook compares the performance of different SVM configurations.

Main Architecture of the Whole Project

The entire project can be understood like this:





⚠ Important Finding

There is an important distinction between **this ZIP project** and the Alzheimer's project you were previously working on.

This ZIP is an **MRI image-based project**.

It uses:

MRI images

↓

Computer Vision

↓

CNN / VGG16 / SVM / KNN / MLP

Whereas your previous Alzheimer's project was based on the **OASIS tabular dataset**, with numerical patient/clinical features.

So if you're planning to use this ZIP for your **final-year project**, we should first decide whether you want:

Option A — Keep this project

MRI-based Alzheimer's classification

MRI → CNN → Dementia Class

Option B — Keep your existing project

Tabular Alzheimer's prediction

Patient Data → ML Models → Alzheimer's Prediction

Option C — Build a stronger combined project

MRI Image Data

+

Clinical/Patient Data

↓

Multimodal ML

↓

Alzheimer's Prediction

↓

Explainable AI

For your current final-year project, I would recommend Option B if you want to stay consistent with the cleaned OASIS tabular dataset you've already been working with. This ZIP is useful as a reference for the **CNN/Grad-CAM image-based approach**, but it is structurally a different implementation.

If you want, I can next **deeply analyze all 6 notebooks cell-by-cell and tell you exactly what each notebook does, what models/algorithms it uses, what dataset paths it expects, and which parts you can reuse for your project.**

yes please

Absolutely. I went through the actual notebook code in the ZIP rather than only the README. Here is the **cell-by-cell technical analysis**, including what is reusable and what needs fixing.

1. Project at a glance

This repository is an **MRI image classification project** for four Alzheimer's/dementia categories:

MildDemented

ModerateDemented

NonDemented

VeryMildDemented

The project contains **5 different modeling approaches + Grad-CAM explainability**:

Notebook	Approach	Main technology
Convolutional Neural Network.ipynb	CNN	TensorFlow/Keras
SVM_Alzheimer.ipynb	VGG16/CNN + SVM	TensorFlow + Scikit-learn
KNN Classifier.ipynb	KNN	Scikit-learn
Ensemble Methods.ipynb	Decision Tree, Bagging, Random Forest	Scikit-learn
Multilayer Perceptron.ipynb	MLP	PyTorch
Grad CAM.ipynb	Explainable AI	TensorFlow/Grad-CAM

2. CNN notebook — the main model

File

Convolutional Neural Network.ipynb

Step 1 — Import libraries

It imports:

numpy

opencv

TensorFlow/Keras

scikit-learn

matplotlib

OpenCV is used for reading MRI images.

Step 2 — Image loading

The function:

load_images_from_folder(folder, label)

does this:

Folder

↓

Read image

↓

Grayscale

↓

Resize to 128×128

↓

Store image

↓

Assign class label

So the model does **not use the original MRI dimensions**.

Everything becomes:

$128 \times 128 \times 1$

Step 3 — Dataset structure

This notebook expects:

dataset/

|

|— MildDemented/

| |— image1.jpg

| |— image2.jpg

| |— ...

|

|— ModerateDemented/

| |— image1.jpg

| |— ...

|

```
└─ NonDemented/  
  └─ image1.jpg  
  └─ ...  
  └─  
└─ VeryMildDemented/  
  └─ image1.jpg  
  └─ ...
```

Labels are:

MildDemented → 0

ModerateDemented → 1

NonDemented → 2

VeryMildDemented → 3

3. Image preprocessing

The images are converted to:

```
all_images = np.array(all_images).reshape(-1,128,128,1) / 255.0
```

So pixel values become:

0 → black

1 → white

instead of:

0 → 255

This is good practice for neural networks.

4. Data augmentation

The CNN uses:

```
ImageDataGenerator(  
    width_shift_range=0.05,  
    height_shift_range=0.05,
```

```
    zoom_range=0.05,  
    horizontal_flip=True  
)
```

So training images can be slightly:

- shifted
- zoomed
- horizontally flipped

This attempts to reduce overfitting.

5. CNN architecture

This is the important part.

The network is:

Input

$128 \times 128 \times 1$

↓

Conv2D 32 filters

↓

MaxPooling

↓

Conv2D 64 filters

↓

MaxPooling

↓

Conv2D 128 filters

↓

MaxPooling

↓

Flatten

↓

Dense 128

↓

Dropout 0.5

↓

Dense 4

↓

Softmax

In simplified form:

MRI

↓

Convolution

↓

Feature extraction

↓

Feature extraction

↓

Feature extraction

↓

Classification

↓

4 classes

The final Dense(4, softmax) produces probabilities for the four classes.

6. Training strategy

The project uses:

StratifiedKFold(n_splits=5)

This means **5-fold cross-validation**.

Conceptually:

Dataset

|

├— Fold 1 → Train 80% / Validate 20%

├— Fold 2 → Train 80% / Validate 20%

├— Fold 3 → Train 80% / Validate 20%

├— Fold 4 → Train 80% / Validate 20%

└— Fold 5 → Train 80% / Validate 20%

This is better than relying on one random train/test split.

Each fold trains for:

150 epochs

using:

Adam optimizer

Categorical Cross Entropy

Accuracy

7. CNN evaluation

For every fold it calculates:

Accuracy

How many images were classified correctly?

Loss

How far predictions were from the actual classes.

Confusion matrix

Predicted

M Mo N VM

Actual Mild ...

Moderate ...

Non ...

VeryMild ...

This is especially useful because accuracy alone doesn't tell you **which dementia classes are being confused**.

8. Important issue with this CNN

There is a methodological issue you should know about.

The notebook combines all images and then performs cross-validation:

all images

↓

StratifiedKFold

If multiple images from the same subject/patient exist, ordinary image-level splitting can potentially put related images into both training and validation sets.

That can make the reported accuracy look better than the model's real-world performance.

Better approach

If patient IDs are available:

Patient-level split

↓

Training patients

+

Validation patients

rather than:

Random image-level split

This is something I would **definitely improve if we're building your final project**.

9. KNN notebook

Now let's look at:

KNN Classifier.ipynb

This notebook is considerably messier.

First, it creates a combined dataset

It expects:

main_data/

|

├─ train/

|

├─ MildDemented/

|

├─ ModerateDemented/

|

├─ NonDemented/

|

└─ VeryMildDemented/

|

└─ test/

├─ MildDemented/

├─ ModerateDemented/

├─ NonDemented/

└─ VeryMildDemented/

Then it combines train and test:

train + test

↓

combined_dataset

⚠ This is a major problem.

After combining the original training and test images, it performs a new:

train_test_split(...)

That means the original test-set separation is lost.

For a rigorous experiment, **you should never combine your test set with training data and then randomly split it again.**

For your improved project, we should keep:

TRAIN

VALIDATION

TEST

completely separate.

10. KNN preprocessing

The KNN notebook performs:

MRI

↓

Grayscale

↓

128×128

↓

CLAHE / adaptive histogram equalization

↓

Thresholding

↓

Flatten

The final image becomes something like:

128×128

flattened into:

16,384 features

So instead of understanding the MRI spatially like CNN does, KNN receives a long numerical vector.

11. KNN model

The actual classifier is:

`KNeighborsClassifier(n_neighbors=5)`

So:

$K = 5$

The algorithm looks at the 5 nearest training samples and determines the class based on their labels.

Flow:

MRI

↓

16,384 numerical features

↓

KNN

↓

5 nearest images

↓

Majority class

↓

Prediction

It also generates:

- accuracy
- confusion matrix
- classification report

12. Ensemble Methods notebook

This notebook experiments with classical ML.

It begins with the same type of image processing:

MRI

↓

Grayscale

↓

128 × 128

↓

Normalize

Then images are flattened for classical ML.

It experiments with:

Decision Tree

Image features



Decision Tree



Class

Bagging

Multiple models



Bootstrap samples



Combine predictions

Random Forest

Many Decision Trees



Combine predictions



Final class

It also uses:

GridSearchCV

to search for better hyperparameters.

That's a good component to reuse.

13. MLP notebook

The MLP notebook is interesting because it switches from TensorFlow to **PyTorch**.

The image is:

$128 \times 128 \times 1$

Then flattened:

16,384 input features

The architecture is:

16,384

↓

512

↓

ReLU

↓

Dropout

↓

128

↓

ReLU

↓

Dropout

↓

4 classes

Specifically:

Linear(16384, 512)

Linear(512, 128)

Linear(128, 4)

The output corresponds to the four classes.

14. MLP training

It uses:

CrossEntropyLoss

and PyTorch:

TensorDataset

DataLoader

with:

batch size = 64

epochs = 150

It records:

Training accuracy

Validation accuracy

and finally generates a confusion matrix.

15. SVM notebook — one of the more interesting parts

The SVM notebook has multiple experiments.

The first important approach is:

MRI

↓

VGG16

↓

Feature Extraction

↓

SVM

This is called **transfer learning + traditional ML classification**.

Instead of training VGG16 to directly classify Alzheimer's, it uses VGG16 to extract useful visual features.

Then SVM performs the final classification.

16. VGG16 + SVM

The basic concept:

MRI

↓

VGG16

↓

High-level image features

↓

Feature vector

↓

SVM

↓

Dementia class

The notebook compares SVM kernels including:

RBF

Polynomial

This is a good idea because SVM performance can vary significantly depending on the kernel.

17. Custom CNN + SVM

The project also tries:

MRI

↓

Custom CNN

↓

Feature extraction

↓

SVM

↓

Prediction

This is actually a pretty interesting hybrid architecture.

Instead of:

CNN → Softmax

it does:

CNN → Features → SVM

The notebook uses cross-validation and compares:

RBF SVM

vs

Polynomial SVM

Then plots fold-by-fold accuracy.

18. Grad-CAM — the explainability component

This is one of the **best parts of the repository conceptually**.

Grad-CAM tries to explain the CNN.

Instead of only saying:

"The model predicts Mild Demented."

it attempts to show:

"These regions of the MRI contributed most to the prediction."

The process is:

MRI

↓

CNN

↓

Prediction

↓

Gradients

↓

Convolutional feature maps

↓

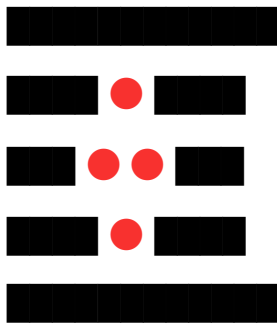
Heatmap

↓

MRI + heatmap

The result is a visualization similar to:

MRI



where the highlighted regions indicate areas influencing the model.

19. Grad-CAM notebook dependency problem

The notebook contains:

```
load_model('alzheimers_CNN_aug_model.h5')
```

But:

alzheimers_CNN_aug_model.h5

is **not included in your ZIP**.

Therefore, the Grad-CAM notebook is not completely standalone.

It needs the previously trained CNN model file.

It also expects convolutional layer names such as:

conv2d_12

conv2d_13

conv2d_14

Those names depend on the exact Keras model that was saved.

So if you retrain the CNN with different layer naming, the Grad-CAM code may break.

20. Major structural problem in the repository

The biggest issue I found is that **different notebooks expect different dataset folders**.

For example:

CNN:

dataset/

while:

KNN:

main_data/combined_dataset/

and:

Ensemble:

Alzheimers/

and:

SVM:

Alzheimer_s Dataset/

So there isn't one standardized dataset structure.

That makes the project difficult to reproduce.

21. Another major problem — dataset isn't included

The ZIP contains:

.ipynb

README.md

but not the actual MRI dataset.

Therefore, someone downloading this repository needs to separately obtain the dataset and arrange it into the correct folder structure.

22. Libraries used across the project

The project uses a fairly large stack:

Python

- |
- | — NumPy
- | — OpenCV
- | — Matplotlib
- | — Seaborn
- | — Scikit-learn
 - | | — SVM
 - | | — KNN
 - | | — Decision Tree
 - | | — Random Forest
 - | | — Bagging
 - | | — Cross Validation
- |
- | — TensorFlow / Keras
 - | | — CNN
 - | | — VGG16
 - | | — Grad-CAM
- |
- | — PyTorch
 - | | — MLP
- |
- | — scikit-image
 - | | — Image preprocessing

23. What I would keep

If we're using this as a reference for your project, I'd keep these ideas:

★ 1. CNN

Definitely keep.

MRI → CNN → 4 classes

★ 2. Transfer Learning

The VGG16 approach is worth keeping as an experiment.

MRI → VGG16 → Features → Classifier

★ 3. SVM

Good classical ML comparison.

★ 4. Random Forest

Useful as a baseline.

★ 5. Confusion Matrix

Definitely keep.

★ 6. Classification Report

Keep:

Precision

Recall

F1-score

Support

★ 7. Grad-CAM

Definitely keep this.

It makes the project much stronger academically because you're adding explainable AI.

24. What I would NOT copy directly

I would not directly reuse:

- ✗ Combining train + test datasets
- ✗ Randomly splitting the combined dataset
- ✗ Different dataset folder names across notebooks

- ✗ 150 epochs without proper early stopping
 - ✗ Relying only on accuracy
 - ✗ Loading an external .h5 model that isn't provided
 - ✗ Patient/image leakage risk
 - ✗ Repeated duplicated preprocessing code
 - ✗ Mixing TensorFlow and PyTorch unless there is a clear reason
 - ✗ Claiming 98% accuracy without carefully validating the split methodology
-

25. How I would rebuild this project

For a clean final-year project, I would restructure it as:

Alzheimer-MRI-Prediction/

```
|
|
| └─ data/
|
|   └─ raw/
|
|   └─ processed/
|
|   └─ train/
|
|   └─ validation/
|
|   └─ test/
|
|
| └─ notebooks/
|
|   └─ 01_EDA.ipynb
|
|   └─ 02_Preprocessing.ipynb
|
|   └─ 03_CNN.ipynb
|
|   └─ 04_Transfer_Learning.ipynb
|
|   └─ 05_Traditional_ML.ipynb
|
|   └─ 06_Explainable_AI.ipynb
|
```

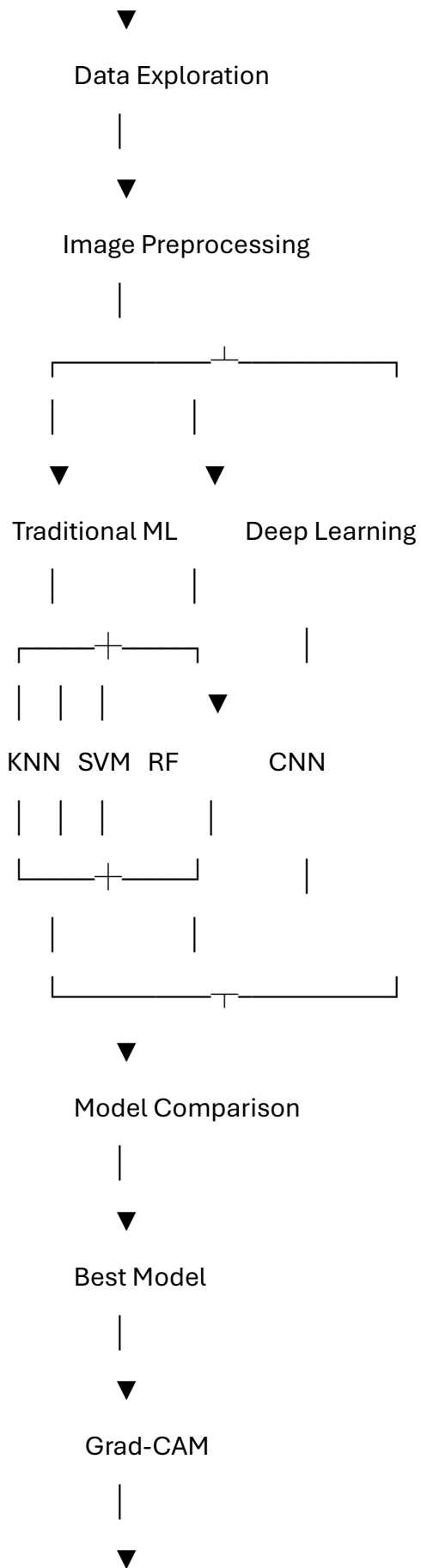
```
├── src/
│   ├── preprocessing.py
│   ├── train.py
│   ├── evaluate.py
│   └── predict.py
│
├── models/
│   ├── cnn.keras
│   └── best_model.keras
│
├── results/
│   ├── confusion_matrix.png
│   ├── training_curve.png
│   ├── model_comparison.png
│   └── gradcam/
│
├── app/
│   └── ...
│
├── requirements.txt
│
└── README.md
```

26. Recommended final architecture

For **your project**, I would make the pipeline:

OASIS MRI Dataset

|



Explainable Result



Prediction Interface

That would give you a much more complete project than simply training one CNN.

🔥 Most important conclusion

The ZIP is actually a **good reference project**, but I would **not submit it unchanged**.

Its strongest ideas are:

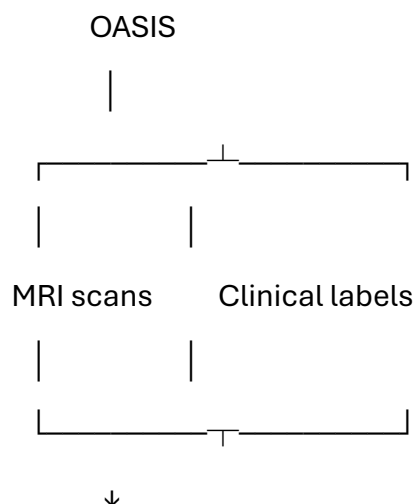
CNN + Transfer Learning + SVM/Random Forest comparison + Grad-CAM.

Its biggest weaknesses are:

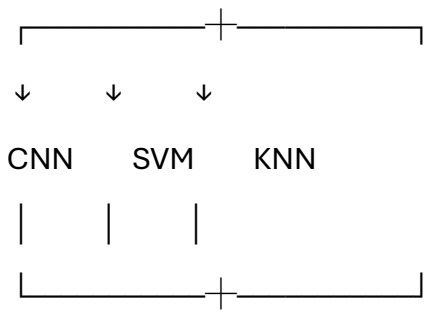
inconsistent dataset paths + test/train mixing in KNN + missing model file + possible data leakage + lack of a clean reproducible project structure.

And there is one very important point for your situation: **this ZIP is an MRI/image-based Alzheimer's project, while your earlier work was on a tabular OASIS dataset.** So we shouldn't blindly merge the two. We can either keep your existing tabular project, switch fully to MRI, or design a genuinely stronger **multimodal Alzheimer's project** combining clinical/tabular information with MRI features.

If your goal is a **final-year project that looks impressive on your resume/GitHub**, I'd choose the third route only if you actually have compatible patient-level data; otherwise, a clean MRI CNN + Grad-CAM project is the safer upgrade.



Machine Learning



CNN SVM KNN



Alzheimer's prediction